

Artifact Overview Document for *Velvet: Parallel Divide-and-Conquer in Safe Rust*

Badia Liokouras, Ben van Werkhoven & Rob V. van Nieuwpoort

This document contains all relevant information to reproduce the results presented in the paper *Velvet: Parallel Divide-and-Conquer in Safe Rust* with the associated artifact. There are two key parts; the first reproduces the tables and figures in the paper using our original data. The second part runs a reduced version of all experiments and produces corresponding tables and figures with the output. This overview document describes the process. All source code and data is provided in a zip file, or can be downloaded separately (see Section 1.3).

1 Getting Started Guide

1.1 Hardware Requirements

The artifact should be able to run on any multi-core x86-64 Linux workstation or cluster node. The original experiments used a dual-socket AMD EPYC 7702 with 128 physical cores. However, this artifact is adjusted for a reduced evaluation, targeting a maximum of 64 cores; more cores are not required.

1.2 Operating System & Software Dependencies

To compile and run the experiments, the following software components are needed. Note that the listed versions do not have to match exactly, as the experiments should be able to run with any modern version of each component.

Component	Version used in original setup
Linux OS	Rocky Linux 9.7 (Blue Onyx)
Rust toolchain	rustc 1.93.1 (stable), Cargo (via rustup)
Clang	20.1.8
Make	GNU Make 4.3
OpenMP	5.1 (via Clang)
OpenCilk	2.1 (built with Clang, Git 2.47.3 and CMake 3.26.5)

To process data and produce the tables and figures in the paper, Python 3.7.13 and the packages `matplotlib`, `numpy`, and `pandas` are required.

1.3 Installation

Python dependencies can be installed using pip: `pip3 install numpy matplotlib pandas`

Rust toolchain can be installed by following [these instructions](#), or simply executing this command: `curl --proto 'https' --tlsv1.2 -sSf https://sh.rustup.rs | sh`

Clang with OpenMP is typically available through the system package manager, though tends to require root access.

OpenCilk can be installed by following [the quick start here](#). After successful installation, set the `$OPENCILK_HOME` environment variable to point to the build folder created during installation: `export OPENCILK_HOME=/path/to/build`

The Artifact is hosted on GitHub; clone and enter the main directory

```
git clone https://github.com/liokouras/velvet-euopar.git ; cd velvet-euopar
```

It is also submitted as a zip file; unzip and enter the folder.

To verify the environment configuration, run the smoke test: `bash smoke_test.sh`
This test checks for each dependency independently and prints either a confirmation or a warning for each. The following are checked:

Python dependencies invoked with `python3`

Rust toolchain invoked with `rustc` and `cargo`, expected to be on `PATH`

Clang with OpenMP invoked with `clang -fopenmp`, expected to be on `PATH`

OpenCilk invoked with `$OPENCILK_HOME/bin/clang -fopencilk`, the `$OPENCILK_HOME` environment variable is expected to be set (not expected to be on `PATH`)

A successful smoke test prints four **VERIFIED** lines; at minimum only Python is needed for data processing, and only Rust is needed to run the basic benchmarks. In case OpenMP and/or OpenCilk cannot be properly configured, the experiments can still run, skipping the corresponding C-versions. This affects the reproduction of Figure 2, which will be missing the corresponding data points.

The smoke test also checks for `lscpu`, which is used by the experiment script to auto-detect the number of physical cores across sockets. If it is not present, the experiments can be launched with an explicit number of cores (see Section 2.2).

1.4 Data Dependencies

The raw output-data from our own experiments are included in the artifact, hosted on GitHub. Most input-data to run the experiments are generated by the benchmarks themselves, except for the Barnes-Hut benchmark. The input for this benchmark is too large to host directly in the repository, so is part of the GitHub release asset. It is downloaded automatically by the relevant runscript; no manual action is required. The download should not take longer than a minute.

2 Step-by-Step Instructions to Reproduce Results

The artifact has the following high-level structure

```
velvet-euopar
├── benchmarks
│   ├── c
│   ├── runscripts
│   └── rust
├── data_processing
│   ├── data
│   └── scripts
├── process_data.sh
├── run_exp.sh
└── smoke_test.sh
```

2.1 Generating Tables and Figures from Raw Data

This part only requires the Python dependencies listed above and should complete within a minute.

Run the following command from inside `velvet-euopar`: `bash process_data.sh`

The above command runs the data-processing pipeline using our original raw data, and produces the last four columns of Table 1 (page 7), Table 2 (page 8), Figure 1 (page 9) and Figure 2 (page 10). The tables are printed to `data_processing/processing_output.txt` and the figures are saved in `data_processing/figs`.

2.2 Running the Benchmarks

This part requires a multi-core Linux machine and, at minimum, the Rust toolchain. OpenMP and OpenCilk are necessary to reproduce the results in Figure 2 of the paper. On our hardware, it takes is approximately 4,5 hours to execute all benchmarks on 64 physical cores.

Run the following command from inside `velvet-euopar`: `bash run_exp.sh [--cores=N]`

The above command runs a **reduced version** (see below) of all our experiments, and then calls the data-processing pipeline using the generated data. It will produce the last four columns of Table 1 (page 7), Table 2 (page 8), Figure 1 (page 9) and, if the C-dependencies are present, Figure 2 (page 10). The tables are printed to `data_processing/processing_output.txt` and the figures are saved in `data_processing/figs`.

The optional `--cores=N` argument overrides the auto-detected physical core count. By default the script detects this automatically via `lscpu`. If `lscpu` is unavailable (indicated by the smoke test), `--cores=N` **must** be provided. In the reduced configuration, the core count is capped at 64 regardless of the value detected or provided. For example, to run the reduced configuration on 32 cores: `bash run_exp.sh --cores=32`.

The default setup of this artifact is reduced compared to that of the paper in the following ways:

Thread counts are capped at 64 (in paper: 128)

Number of runs per data point is 3, where the first is discarded (in paper: 6, where the first is discarded)

Problem sizes are adjusted for each benchmark and displayed below, to be compared with the second column of Table 1 in the paper. Serial thresholds remain unchanged.

Benchmark	<i>fib</i>	<i>adapint</i>	<i>tsp</i>	<i>nqueen</i>	<i>bh</i>	<i>matmul</i>	<i>sort</i>
Problem size	$n = 48$	$[a, b] = [0, 8 \cdot 10^5]$ $\epsilon = 10^{-4}$	Nr. cities: 18	$N = 15$	Nr. bodies: 10^6	Mat. dim: 4096×4096	Array size: 10^9

Total runtime on our hardware is approx. 4,5 hours (in paper: approx. 48 hours)

In addition to the above reductions, we also omit the **Rayon**, **Demo-Strassen** version of the *matmul* benchmark, because it takes too long to execute. This affects only one line in the *matmul* plot of Figure 1.

The full original experiments can also be run using the following command, but we emphasize that this will likely take days to complete: `bash run_exp.sh full`

2.3 Interpreting the Results

The output of **2.1** should match the paper’s tables and figures exactly (same raw data) with the exception that Table 2 is transposed in the paper for better layout and spacing.

The output of **2.2** will naturally differ from the absolute values reported in the paper, first and foremost due to the reduced configuration, and due to differing hardware. The following trends should be reproduced:

Table 1 This is basic information about each benchmark, no specific trends expected.

Table 2 Work overhead factors close to 1.00 *except* for Fib with no threshold, which will have a large overhead

Figure 1 Near-linear speedup for the benchmarks in the top row, and sub-linear scaling for the bottom row. Velvet and Rayon perform comparatively. There will be no line for **Rayon**, **Demo Strassen**.

Figure 2 Similar trends as in Figure 1, with Velvet performing comparatively with the C frameworks. If any of the C-dependencies are missing, the corresponding data will be absent from this figure.